AIN SHAMS UNIVERSITY

FACULTY OF ENGINEERING

Assignment 1

MicroGrad: Scalar Automatic Differentiation Engine

Course	CSE485–CSE488: Deep Learning
Student	Ahmed Walid Elsayed
Student ID	23P0038
Submission	Assignment 1 — MicroGrad

Mathematical derivations, implementation design, and experimental results

1. Objective and Overview

The objective of this assignment is to build a small scalar automatic differentiation engine inspired by MicroGrad. The engine records scalar computations in a computational graph and computes derivatives using the chain rule and reverse-mode backpropagation. A small neural-network API is then built on top of this engine.

The implementation is divided into two modules:

engine.py implements the Value class, arithmetic operators, activation functions, local derivative rules, topological sorting, and the backward pass.

nn.py implements Neuron, Layer, and MLP classes, including random parameter initialization, forward passes, parameter collection, and gradient reset.

2. Computational Graph and Value Objects

Each Value object stores a scalar data value and the gradient accumulated for that value. It also stores its predecessor nodes in `_prev`, the operation label in `_op`, and a local backward function in `_backward`. The optional label is useful for debugging and graph inspection.

When an operation is performed, the engine computes its numerical result immediately (the forward pass) and creates a new Value whose predecessor set records the inputs to that operation. The new node's backward function encodes the local derivative rule.

```
a = Value(2.0)
b = Value(3.0)
c = a * b       # c.data == 6.0
# c remembers that it was produced from a and b
```

The graph records dependencies rather than storing a manually written formula for the entire final expression. During backpropagation, each operation contributes gradient values to its input nodes.

3. Local Derivative Rules

The backward function for each operation applies its local derivative and multiplies by the gradient arriving from the output. This is the chain rule in reverse-mode automatic differentiation.

3.1 Addition

For z = a + b, the local derivatives are ∂z/∂a = 1 and ∂z/∂b = 1. If the upstream gradient is ∂L/∂z, then each input receives that same contribution:

∂L/∂a += ∂L/∂z; ∂L/∂b += ∂L/∂z

3.2 Multiplication

For z = ab, the product rule gives ∂z/∂a = b and ∂z/∂b = a. Therefore:

∂L/∂a += b · ∂L/∂z; ∂L/∂b += a · ∂L/∂z

3.3 Power

For z = a^n, where n is an integer or floating-point scalar exponent, the power rule gives ∂z/∂a = n a^(n–1). The implementation multiplies this local derivative by the upstream gradient:

∂L/∂a += n · a^(n–1) · ∂L/∂z

3.4 Negation, subtraction, and division

Negation is implemented as multiplication by –1. Subtraction is implemented using addition and negation: a – b = a + (–b). Division is implemented using the reciprocal power: a / b = a · b^(–1). Consequently, these operators reuse the multiplication and power derivative rules. Division by zero is checked and raises ZeroDivisionError.

3.5 Hyperbolic tangent

For z = tanh(a), the derivative is ∂z/∂a = 1 – tanh²(a). If t = tanh(a), the backward contribution is:

∂L/∂a += (1 – t²) · ∂L/∂z

The forward value is calculated using the exponential identity tanh(a) = (e^(2a) – 1)/(e^(2a) + 1).

3.6 Exponential

For z = e^a, the derivative is ∂z/∂a = e^a = z. The implementation therefore adds z multiplied by the upstream gradient to the input gradient:

∂L/∂a += e^a · ∂L/∂z

4. Reverse-Mode Backpropagation

The backward pass begins at the final output node. Its gradient is initialized to 1 because the derivative of the output with respect to itself is 1. The engine then visits nodes in reverse topological order and calls each node's local backward function.

4.1 Topological sorting using depth-first search

The _get_topo method performs a depth-first search (DFS). It maintains a visited set so that each node is processed only once. For each node, it first recursively visits all predecessor nodes and only then appends the current node to the topological list. This post-order traversal ensures that predecessors appear before the node that depends on them.

```
topo = []
visited = set()

def build_topo(node):
    if node not in visited:
        visited.add(node)
        for child in node._prev:
            build_topo(child)
        topo.append(node)

build_topo(output)
return topo
```

4.2 Applying the backward pass

After obtaining the topological order, the engine sets the output gradient to 1.0 and iterates over the reversed list. Each node's _backward function propagates its gradient to its predecessors.

```
topo = self._get_topo()
self.grad = 1.0

for node in reversed(topo):
    node._backward()
```

4.3 Why gradients are accumulated

The implementation uses += rather than assignment when propagating gradients. A node may influence the output through more than one path. In that case, the total derivative is the sum of the contributions from all paths. The visited set in DFS prevents repeated traversal of a shared node, while gradient accumulation preserves all derivative contributions.

For example, if c = a + a, then ∂c/∂a = 1 + 1 = 2. The two contributions must be added to the same gradient.

5. Neural Network API

5.1 Neuron

A Neuron with n inputs stores n weights and one bias. Each parameter is represented as a Value object and initialized independently from a uniform distribution between –1 and 1. For an input vector x, the pre-activation is:

u = Σ■ w■x■ + b

The neuron returns tanh(u). This gives the neuron a nonlinear activation function and allows a network of neurons to represent nonlinear relationships.

5.2 Layer

A Layer contains nout neurons, each receiving nin inputs. Its forward pass evaluates every neuron on the same input vector. If the layer contains one neuron, it returns that single Value; otherwise, it returns a list of Values.

5.3 Multi-Layer Perceptron

The MLP constructs a sequence of layers from the input size and the supplied output sizes. For example, MLP(nin=2, nouts=[4, 1]) creates a 2 → 4 → 1 network: four neurons in the hidden layer and one neuron in the output layer.

The parameter count for this architecture is 4 × (2 weights + 1 bias) + 1 × (4 weights + 1 bias) = 12 + 5 = 17 trainable parameters.

5.4 Parameter collection and gradient reset

The parameters() methods flatten the weights and biases from neurons, layers, and the full MLP into one list. The zero_grad() methods call reset_grad() on every parameter so that gradients from a previous training iteration do not unintentionally carry into the next iteration.

6. Training Demonstration: XOR

The notebook demonstrates the implementation on the XOR dataset. XOR returns a positive class when its two binary inputs differ and a negative class when they are equal. The targets are encoded as –1 and +1 to match the range of the tanh output.

Input	Target
[0, 0]	–1
[0, 1]	+1
[1, 0]	+1
[1, 1]	–1

6.1 Loss function

The notebook uses mean squared error (MSE) over the four examples:

MSE = (1/N) Σ■ (■■ – y■)²

6.2 Gradient descent

For each training iteration, the model performs a forward pass to calculate the loss, resets parameter gradients, calls backward() on the loss, and updates each parameter using gradient descent:

parameter ← parameter – learning_rate × gradient

The demonstration uses a learning rate of 0.05 and 5,000 training epochs. The model is initialized with a fixed random seed to make the run reproducible under the same Python and execution environment.

6.3 Observed results

The completed notebook run reported a loss of 1.121355 at epoch 0 and 0.001264 at epoch 4,500. The final predictions were:

Input	Target	Prediction	Class correct?
[0, 0]	–1	–0.9750	Yes

[0, 1]	+1	+0.9609	Yes
[1, 0]	+1	+0.9715	Yes
[1, 1]	–1	–0.9618	Yes

All four examples were classified correctly using a zero threshold on the model output, giving 100% accuracy on this four-example demonstration dataset. This result demonstrates that the implemented nonlinear MLP can learn the XOR mapping for this training run. It is a demonstration result on the training examples, not a claim about generalization to a separate test set.

7. Testing and Verification

The provided pytest suite was executed after implementing both modules. All 12 supplied tests passed: nine tests for the scalar engine and three tests for the neural-network API.

Test group	Tests passed	Result
engine.py	9 / 9	PASS
nn.py	3 / 3	PASS
Total	**12 / 12**	**PASS**

The tests cover Value initialization and representation, arithmetic operations, type handling, powers, activation functions, gradient accumulation in a shared graph, chain-rule backpropagation, division by zero, type casting, and the Neuron, Layer, and MLP APIs.

8. Design Justifications

Scalar Value abstraction: Storing one scalar and its graph metadata per Value keeps the engine small and makes each local derivative rule explicit.

Operator overloading: Python arithmetic operators allow ordinary-looking expressions to build the computation graph while preserving the underlying Value objects.

Local backward closures: Each output node stores the derivative rule associated with the operation that created it. This keeps the backward implementation modular.

Topological order: DFS ensures that gradient contributions from downstream operations are available before a node propagates its gradient to its own predecessors.

Gradient accumulation: Using += correctly handles graphs where a value is reused in multiple parts of an expression.

Small neural-network API: Neuron, Layer, and MLP separate individual computation, grouping, and network composition. Parameter collection and gradient reset make the training loop concise.

9. Conclusion

The assignment implementation provides a scalar reverse-mode automatic differentiation engine and a basic neural-network API. The engine records computational dependencies, computes local derivatives, and propagates gradients through a DFS-derived topological order. The MLP built on top of the engine was trained on the XOR demonstration, achieving 100% classification accuracy on the four training examples in the reported

run. All 12 supplied automated tests passed.

Submission note: This report documents the implementation and experiment results recorded during the assignment workflow. Before submission, ensure the notebook and source files in the ZIP are the same versions used to generate these results.